

Unit 01: Introduction

CONTENTS

Objectives

Introduction

- 1.1 Data Structure
- 1.2 Data Structure Operations
- 1.3 Abstract Data Type
- 1.4 Algorithm
- 1.5 Characteristics of an Algorithm
- 1.6 Types of Algorithms
- 1.7 Algorithm Complexity
- 1.8 Asymptotic Notations

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Question

Further Readings

Objectives

After studying this unit, you will be able to:

- Describe basic concepts of data structure
- Learn Algorithm and its complexity
- Know Abstract data type
- Data structure types

Introduction

The static representation of a linear ordered list using an array wastes resources and, in some situations, causes overflows. We no longer want to pre-allocate memory to any linear list; instead, we want to allocate memory to elements as they are added to the list. This necessitates memory allocation that is dynamic.

Semantically data can exist in either of the two forms – atomic or structured. In most of the programming problems data to be read, processed and written are often related to each other. Data items are related in a variety of different ways. Whereas the basic data types such as integers, characters etc. can be directly created and manipulated in a programming language, the responsibility of creating the structured type data items remains with the programmers themselves. Accordingly, programming languages provide mechanism to create and manipulate structured data items.

A data structure is a type of storage that is used to organize and store data. It is a method of organizing data on a computer so that it may be easily accessible and modified.

It's critical to choose the correct data format for your project based on your requirements and project. If you wish to store data sequentially in memory, for example, you can use the Array data structure.

1.1 Data Structure

A data structure is a set of data values along with the relationship between the data values. Since, the operations that can be performed on the data values depend on what kind of relationships exists among them, we can specify the relationship amongst the data values by specifying the operations permitted on the data values. Therefore, we can say that a data structure is a set of values along with the set of operations permitted on them. It is also required to specify the semantics of the operations permitted on the data values, and this is done by using a set of axioms, which describes how these operations work, and therefore a data structure is made of:

1. A set of data values.
2. A set of functions specifying the operations permitted on the data values.
3. A set of axioms describing how these operations work.

Hence, we conclude that a data structure is a triple (D, F, A) , where

1. D is a set of data values
2. F is a set of functions
3. A is a set of axioms

A triple (D, F, A) is referred to as an abstract data structure because it does not tell anything about its actual implementation. It does not tell anything about how these values will be physically represented in the computer memory and these functions will be actually implemented.

Therefore, every abstract data structure is required to be implemented, and the implementation of an abstract data structure requires mapping of the abstract data structure to be implemented into the data structure supported by the computer. For example, if the abstract data structure to be implemented is integer, then it can be implemented by mapping into bits which is a data structure supported by hardware. This requires that every integer data value is to be represented using suitable bit patterns and expressing the operations on integer data values in terms of operations for manipulating bits.

Data Structure mainly two types:

1. Linear type data structure
2. Non-linear type data structure

Linear data structure: A linear data structure traverses the data elements sequentially, in which only one data element can directly be reached. Ex: Arrays, Linked Lists

Arrays: An array is a collection of similar type of data items and each data item is called an element of the array.

The data type of the element may be any valid data type like char, int, float or double. — The individual elements of the array are: — `age[0]`, `age[1]`, `age[2]`, `age[3]`, `age[98]`, `age[99]`.

Linked List: Linked list is a linear data structure which is used to maintain a list in the memory.

It can be seen as the collection of nodes stored at non-contiguous memory locations. Each node of the list contains a pointer to its adjacent node

Stack: Stack is a linear list in which insertion and deletions are allowed only at one end, called top.

A stack is an abstract data type, can be implemented in most of the programming languages. It is named as stack because it behaves like a real-world stack, for example: - piles of plates or deck of cards etc.

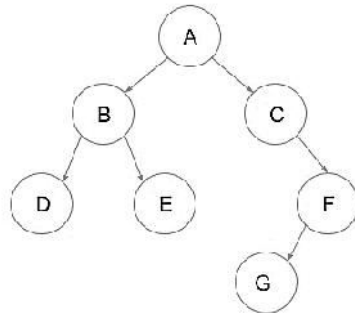
Queue is a linear list in which element can be inserted only at one end called rear and deleted only at other end called front.

It is abstract data structure, similar to stack. It is open at both end therefore it follows first-in-first-out (FIFO) technique for storing the data items.

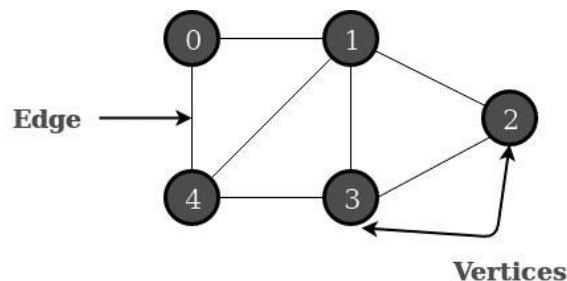
Non-linear data structure: Every data item is attached to several other data items in a way that is specific for reflecting relationships. The data items are not arranged in a sequential structure.

Ex: Trees, Graphs.

Trees: Trees are multilevel data structures with a hierarchical relationship among its elements known as nodes.



Graphs: Graphs can be defined as the pictorial representation of the set of elements (represented by vertices) connected by the links known as edges



Basic Terminology

Data: Data can be defined as an elementary value or the collection of values, for example, student's name and its id are the data about the student.

Group Items: Data items which have subordinate data items are called Group item, for example, name of a student can have first name and the last name.

Record: Record can be defined as the collection of various data items, for example, if we talk about the student entity, then its name, address, course and marks can be grouped together to form the record for the student.

Field: A File is a collection of various records of one type of entity, for example, if there are 60 students in the class, then there will be 20 records in the related file where each record contains the data about each student.

Need of Data Structures

As applications are getting complex and amount of data is increasing day by day, there may arise many problems:

Processor speed: To handle very large amount of data, high speed processing is required, but as the data is growing day by day to the billions of files per entity, processor may fail to deal with that much amount of data.

Data Search: Consider an inventory size of 100 items in a store, If our application needs to search for a particular item, it needs to traverse 100 items every time, results in slowing down the search process.

Multiple requests: If thousands of users are Searching the data simultaneously on a web server, then there are the chances that a very large server can be failed during that process To solve these problems data structures are used.

Basic Concept of Data

The memory (also called storage or core) of a computer is simply a group of bits (switches). At any instant of the computer's operation any particular bit in memory is either 0 or 1 (off or on).

The setting or state of a bit is called its value and that is the smallest unit of information. A set of bit values form data.

Some logical properties can be imposed on the data. According to the logical properties data can be segregated into different categories. Each category having unique set of logical properties is known as data type.

Data type are of two types:

1. Simple data type or elementary item like integer, character.
2. Composite data type or group item like array, structure, union.

Data structures are of two types:

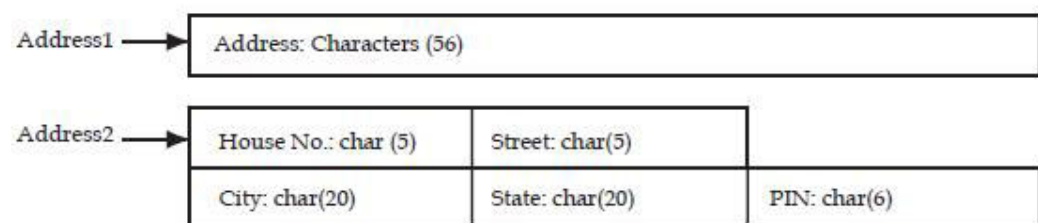
1. *Primitive Data Structures*: Data can be structured at the most primitive level, where they are directly operated upon by machine-level instructions. At this level, data may be character or numeric, and numeric data may consist of integers or real numbers.
2. *Non-primitive Data Structures*: Non-primitive data structures can be classified as arrays, lists, and files.

An array is an ordered set which contains a fixed number of objects. No deletions or insertions are performed on arrays i.e. the size of the array cannot be changed. At best, elements may be changed.

A list, by contrast, is an ordered set consisting of a variable number of elements to which insertions and deletions can be made, and on which other operations can be performed. When a list displays the relationship of adjacency between elements, it is said to be linear; otherwise, it is said to be non-linear.

A file is typically a large list that is stored in the external memory of a computer. Additionally, a file may be used as a repository for list items (records) that are accessed infrequently.

From a real world perspective, very often we have to deal with structured data items which are related to each other. For instance, let us consider the address of an employee. We can take address to be one variable of character type or structured into various fields, as shown below:



As shown above Address1 is unstructured address data. In this form you cannot access individual items from it. You can at best refer to the entire address at one time. While in the second form, i.e., Address2, you can access and manipulate individual fields of the address – House No., Street, PIN etc. Given hereunder are two instances of the address1 and address2 variables.

1.2 Data Structure Operations

The data appearing in our data structure is processed by means of certain operations. The particular data structure that one chooses for a given situation depends largely on the frequency with which specific operations are performed. The following four operations play a major role:

1. **Traversing**: Accessing each record exactly once so that certain items in the record may be processed. (This accessing or processing is sometimes called 'visiting' the records.)
2. **Searching**: Finding the location of the record with a given key value, or finding the locations of all records, which satisfy one or more conditions.

3. Inserting: Adding new records to the structure.

4. Deleting: Removing a record from the structure.

Sometimes two or more data structure of operations may be used in a given situation; e.g., we may want to delete the record with a given key, which may mean we first need to search for the location of the record.

1.3 Abstract Data Type

Before we move to abstract data type let us understand what data type is. Most of the languages support basic data types viz. integer, real, character etc. At machine level, the data is stored as strings containing 1's and 0's. Every data type interprets the string of bits in different ways and gives different results. In short, data type is a method of interpreting bit patterns.

Every data type has a fixed type and range of values it can operate on. For example, an integer variable can hold values between the min and max values allowed and carry out operations like addition, subtraction etc. For character data type, the valid values are defined in the character set and the operations performed are like comparison, conversion from one case to another etc.

There are fixed operations, which can be carried out on them. We can formally define data types as a formal description of the set of values and operations that a variable of a given type may take. That was about the inbuilt data types. One can also create user defined data types, decide the range of values as well as operations to be performed on them. The first step towards creating a user defined data type or a data structure is to define the logical properties. A tool to specify the logical properties of a data type is Abstract Data Type.

Data abstraction can be defined as separation of the logical properties of the organization of programs' data from its implementation. This means that it states what the data should be like.

It does not consider the implementation details. ADT is the logical picture of a data type; in addition, the specifications of the operations required to create and manipulate objects of this data type.

While defining an ADT, we are not concerned with time and space efficiency or any other implementation details of the data structure. ADT is just a useful guideline to use and implement the data type.

An ADT has two parts:

1. Value definition
2. Operation definition.

Value definition is again divided into two parts:

1. Definition clause
2. Condition clause

As the name suggests the definition clause states the contents of the data type and condition clause defines any condition that applies to the data type. Definition clause is mandatory while condition clause is optional.

In operation definition, there are three parts:

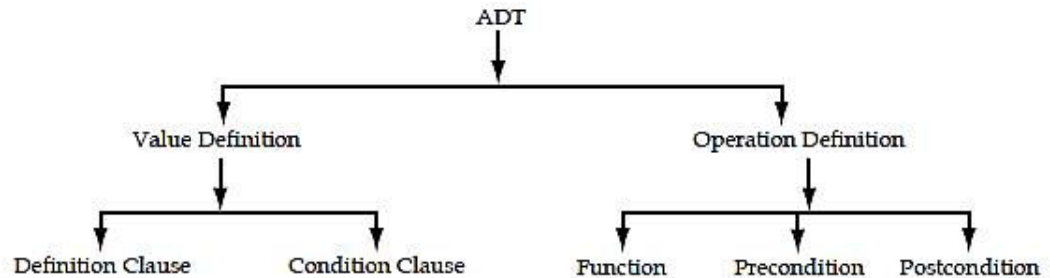
1. Function
2. Precondition
3. Postcondition

The *function* clause defines the role of the operation. If we consider the addition operation on integers the function clause will state that two integers can be added using this function. In general, precondition specifies any restrictions that must be satisfied before the operation can be applied.

This clause is optional. If we consider the division operation on integers then the precondition will state that the divisor should not be zero. So any call for divide operation, which does not satisfy this condition, will not give the desired output.

Precondition specifies any condition that may apply as a pre-requisite for the operation definition. There are certain operations that can be carried out if certain conditions are satisfied. For example, in case of division operation the divisor should never be equal to zero. Only if this condition is satisfied the division operation is carried out. Hence, this becomes a precondition. In that case &(ampersand) should be mentioned in the operation definition.

Postcondition specifies what the operation does. One can say that it specifies the state after the operation is performed. In the addition operation, the post condition will give the addition of the two integers.



Component of ADT

As an example, let us consider the representation of integer data type as an ADT. We will consider only two operations addition and division.

Value Definition

1. Definition clause: The values must be in between the minimum and maximum values specified for the particular computer.
2. Condition clause: Values should not include decimal point.

Operations

1. add (a, b)

Function: add the two integers a and b.

Precondition: no precondition.

Postcondition: output = a + b

2. Div (a, b)

Function: Divide a by b.

Precondition: $b \neq 0$

Postcondition: output = a/b .

There are two ways of implementing a data structure viz. static and dynamic. In static implementation, the memory is allocated at the compile time. If there are more elements than the specified memory then the program crashes. In dynamic implementation, the memory is allocated as and when required during run time.

Any type of data structure will have certain basic operations to be performed on its data like insert, delete, modify, sort, search etc. depending on the requirement. These are the entities that decide the representation of data and distinguish data structures from each other.

Let us see why user defined data structures are essential. Consider a problem where we need to create a list of elements. Any new element added to the list must be added at the end of the list and whenever an element is retrieved, it should be the last element of the list. One can compare this to a pile of plates kept on a table. Whenever one needs a plate, the last one on the pile is taken and if a plate is to be added on the pile, it will be kept on the top. The description wants us to implement a stack. Let us try to solve this problem using arrays.

We will have to keep track of the index of the last element entered in the list. Initially, it will be set to -1. Whenever we insert an element into the list, we will increment the index and insert the value into the new index position. To remove an element, the value of current index will be the output

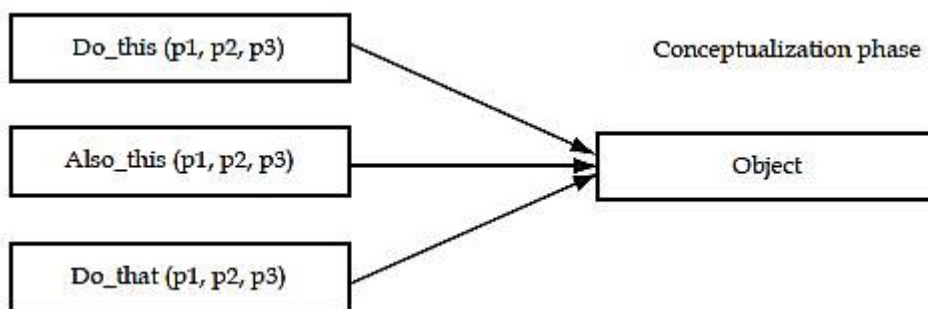
and the index will be decremented by one. In the above representation, we have satisfied the insertion and deletion conditions.

Using arrays we could handle our data properly, but arrays do allow access to other values in addition to the top most one. We can insert an element at the end of the list but there is no way to ensure that insertion will be done only at the end. This is because array as a data structure allows access to any of its values. At this point we can think of another representation, a list of elements where one can add at the end, remove from the end and elements other than the top one are not accessible. As already discussed, this data structure is called as STACK. The insertion operation is known as push and removal as pop. You can try to write an ADT for stacks.

Another situation where we would like to create a data structure is while working with complex numbers. The operations add, subtract division and multiplication will have to be created as per the rules of complex numbers. The ADT for complex numbers is given below. Only addition and multiplication operations are considered here, you can try to write the remaining operations.

Abstract Data Type (ADT)

1. A framework for an object interface
2. What kind of stuff it'd be made of (no details)?
3. What kind of messages it would receive and kind of action it'll perform when properly triggered?



From this we figure out

1. Object make-up (in terms of data)
2. Object interface (what sort of messages it would handle?)
3. How and when it should act when triggered from outside (public trigger) and by another object friendly to it?

These concerns lead to an ADT – a definition for the object.

An Abstract Data Type (ADT) is a set of data items and the methods that work on them.

An implementation of an ADT is a translation into statements of a programming language, of the declaration that defines a variable to be of that ADT, plus a procedure in that language for each operation of the ADT. An implementation chooses a data structure to represent the ADT; each data structure is built up from the basic data types of the underlying programming language.

Thus, if we wish to change the implementation of an ADT, only the procedures implementing the operations would change. This change would not affect the users of the ADT.

Although the terms 'data type', 'data structure' and 'abstract data type' sound alike, they have different meanings. In a programming language, the data type of a variable is the set of values that the variable may assume. For example, a variable of type Boolean can assume either the value true or the value false, but no other value. An abstract data type is a mathematical model, together with various operations defined on the model. As we have indicated, we shall design algorithms in terms of ADTs, but to implement an algorithm in a given programming language.

we must find some way of representing the ADTs in terms of the data types and operators supported by the programming language itself. To represent the mathematical model underlying an ADT, we use data structures, which are a collection of variables, possibly of several data types, connected in various ways.

The cell is the basic building block of data structures. We can picture a cell as a box that is capable of holding a value drawn from some basic or composite data type. Data structures are created by giving names to aggregates of cells and (optionally) interpreting the values of some cells as representing relationships or connections (e.g., pointers) among cells.

1.4 Algorithm

Algorithm is set of rules/ instructions that step-by-step define how a work is to be executed upon in order to get the expected results.

systematic procedure that produces in a finite number of steps the answer to a question or the solution of a problem.

Computer algorithms work via input and output. They take the input and apply each step of the algorithm to that information to generate an output.

E.g. a search engine is an algorithm that takes a search query as an input and searches its database for items relevant to the words in the query. It then outputs the results.

Financial companies use algorithms in areas such as loan pricing, stock trading, asset-liability management, and many automated functions. For example, algorithmic trading, known as algo trading, is used for deciding the timing, pricing, and quantity of stock orders. Also referred to as automated trading or black-box trading, algo trading uses computer programs to buy or sell securities at a pace not possible for humans.

Computer algorithms make life easier by trimming the time it takes to manually do things. In the world of automation, algorithms allow workers to be more proficient and focused. Algorithms make slow processes more proficient. In many cases, especially in automation, algos can save companies money.

1.5 Characteristics of an Algorithm

- Well defined Input and output
- Clear and Unambiguous
- Finite-ness
- Feasible
- Language Independent

Input and output should be defined precisely.

Each step in the algorithm should be clear and unambiguous.

Algorithms should be most effective among many different ways to solve a problem.

An algorithm shouldn't include computer code. Instead, the algorithm should be written in such a way that it can be used in different programming languages.

The algorithm must be finite, i.e. it should not end up in an infinite loops or similar.

The algorithm must be simple, generic and practical, such that it can be executed upon will the available resources. It must not contain some future technology, or anything.

The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be same, as expected.

1.6 Types of Algorithms

Algorithms are categorized based on the concepts that they use to accomplish a task.

- Divide and conquer algorithms
- Brute force algorithms
- Greedy algorithms
- Backtracking algorithms
- Randomized algorithms



Example:

Step 1: Start

Step 2: Declare variables num1, num2 and sum.

Step 3: Read values num1 and num2.

Step 4: Add num1 and num2 and assign the result to sum.

Sum = num1+num2

Step 5: Display sum

Step 6: Stop

1.7 Algorithm Complexity

Space Complexity

Time Complexity

Space Complexity: Space complexity of an algorithm refers to the amount of memory that this algorithm requires to execute and get the result. This can be for inputs, temporary operations, or outputs.

Fixed Part: This refers to the space that is definitely required by the algorithm. For example, input variables, output variables, program size, etc.

Variable Part: This refers to the space that can be different based on the implementation of the algorithm. For example, temporary variables, dynamic memory allocation, recursion stack space, etc.

Time Complexity: Time complexity of an algorithm refers to the amount of time that this algorithm requires to execute and get the result. This can be for normal operations, conditional if-else statements, loop statements, etc.

Constant time part: Any instruction that is executed just once comes in this part. For example, input, output, if-else, switch, etc.

Variable Time Part: Any instruction that is executed more than once, say n times, comes in this part. For example, loops, recursion, etc.

1.8 Asymptotic Notations

To measure the efficiency of an algorithm asymptotic analysis is used.

The efficiency of an algorithm depends on the amount of time, storage and other resources required to execute the algorithm.

Performance of algorithm is change with different type of inputs.

The study of change in performance of the algorithm with the change in the order of the input size is defined as asymptotic analysis.

Asymptotic notations are the mathematical notations used to describe the running time of an algorithm when the input tends towards a particular value or a limiting value.

Types of asymptotic notations

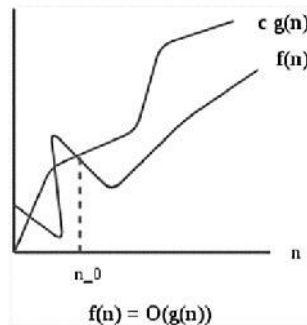
There are three major asymptotic notations

- Big-O notation
- Omega notation
- Theta notation

Big-O notation represents the upper bound of the running time of an algorithm. It gives the worst-case complexity of an algorithm.

$O(n)$ is useful when we only have an upper bound on the time complexity of an algorithm.

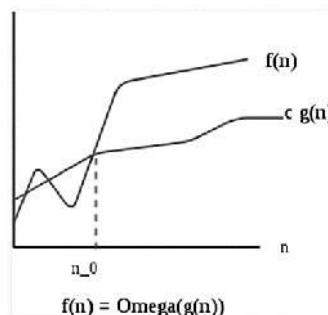
It is widely used to analyse an algorithm as we are always interested in the worst-case scenario.



$$O(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0 \}$$

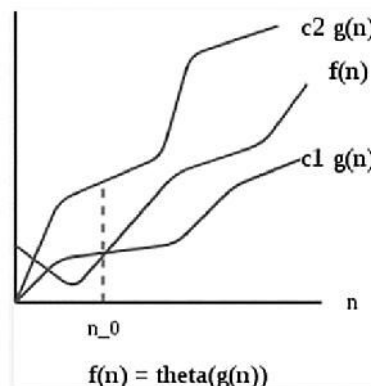
Omega notation represents the lower bound of the running time of an algorithm. It provides the best-case complexity of an algorithm.

Omega Notation can be useful when we have lower bound on time complexity of an algorithm. Omega notation is the least used notation among all three.



$$\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0 \}$$

Theta notation encloses the function from above and below. It represents the upper and the lower bound of the running time of an algorithm, it is used for analysing the average-case complexity of an algorithm.



$$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0 \}$$

Properties of Asymptotic Notations

If $f(n)$ is $O(g(n))$ then $a \cdot f(n)$ is also $O(g(n))$; where a is a constant.

General Properties

If $f(n)$ is $O(g(n))$ then $a \cdot f(n)$ is also $O(g(n))$; where a is a constant.

Transitive Properties

If $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$ then $f(n) = O(h(n))$

Reflexive Properties

If $f(n)$ is given then $f(n)$ is $O(f(n))$

Symmetric Properties

If $f(n)$ is $\Theta(g(n))$ then $g(n)$ is $\Theta(f(n))$

Transpose Symmetric Properties

If $f(n)$ is $O(g(n))$ then $g(n)$ is $\Omega(f(n))$

Summary

- Data Structure is method or technique to data organization, management, and storage format in the computer so we can perform operations on the stored data more efficiently.
- Data structure is a combination of one or more basic data types to form a single addressable data type.
- An algorithm is a finite set of instructions which, when followed, accomplishes a particular task, the termination of which is guaranteed under all cases, i.e. the termination is guaranteed for every input.
- The instructions must be unambiguous and the algorithm must produce the output within a finite number of executions of its instructions.
- Abstract data type (ADT) is a mathematical model with a collection of operations defined on that model. Although the terms 'data type', 'data structure' and 'abstract data type' sound alike, they have different meanings.

Keywords

- *Data*: Data can be defined as an elementary value or the collection of values, for example, student's name and its id are the data about the student.
- *Group Items*: Data items which have subordinate data items are called Group item, for example, name of a student can have first name and the last name.
- *Linear Data Structure*: A linear data structure traverses the data elements sequentially, in which only one data element can directly be reached.
- *Non-linear Data Structure*: Every data item is attached to several other data items in a way that is specific for reflecting relationships. The data items are not arranged in a sequential structure.
- *Searching*: Finding the location of the record with a given key value, or finding the locations of all records, which satisfy one or more conditions.
- *Traversing*: Accessing each record exactly once so that certain items in the record may be processed.

Self Assessment

1. Which is type of data structure.

- A. Primitive
 - B. Non-primitive
 - C. Both primitive and non-primitive
 - D. None of above
2. Which of the following is linear data structure?
- A. Trees
 - B. Arrays
 - C. Graphs
 - D. None of these
3. Which of the following is non-linear data structure?
- A. Array
 - B. Linked lists
 - C. Stacks
 - D. None of these
4. User defined data type is also called?
- A. Primitive
 - B. Identifier
 - C. Non-primitive
 - D. None of these
5. Stack is based on which principle
- A. FIFO
 - B. LIFO
 - C. Push
 - D. None of the Above
6. What are the characteristics of an Algorithm.
- A. Clear and Unambiguous
 - B. Finite-ness
 - C. Feasible
 - D. All of above
7. A procedure for solving a problem in terms of action and their order is called as
- A. Program instruction
 - B. Algorithm
 - C. Process
 - D. Template
8. Algorithm can be represented as

- A. Pseudocode
- B. Flowchart
- C. None of the above
- D. Both Pseudocode and Flowchart

9. What are the different types of Algorithms?

- A. Brute force algorithms
- B. Greedy algorithms
- C. Backtracking algorithms
- D. All of these

10. Which is algorithm complexity.

- A. Space Complexity
- B. Time Complexity
- C. Both space and time complexity
- D. None of aboveone of these

11. Which one is asymptotic notations?

- A. Big-O notation
- B. Omega notation
- C. Theta notation
- D. All of above

12. Big-O Notation represents...

- A. Space complexity
- B. Upper bound of the running time of an algorithm
- C. Lower bound of the running time of an algorithm
- D. None of above

13. Omega Notation (Ω -notation) represents....

- A. Upper bound of the running time of an algorithm
- B. Space complexity
- C. Lower bound of the running time of an algorithm
- D. None of above

14. Which is property of Asymptotic Notations?

- A. Reflexive
- B. Symmetric
- C. Transpose Symmetric
- D. All of these

15. Abstract Data Type having.

- A. Value definition
- B. Operation definition
- C. Both value and operation definition
- D. None of above.

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. C | 2. B | 3. D | 4. C | 5. B |
| 6. D | 7. B | 8. D | 9. D | 10. C |
| 11. D | 12. B | 13. C | 14. D | 15. C |

Review Question

1. Define data structure and its application.
2. What are the advantages of data structure?
3. Discuss abstract data type.
4. What is significance of space and time complexity in algorithm.
5. Explain different types of algorithms.
6. Discuss Asymptotic notations with example.
7. Define record and file.



Further Readings

- Data Structures and Algorithms; Shi-Kuo Chang; World Scientific c.
- Data Structures and Efficient Algorithms, Burkhard Monien, Thomas Ottmann, Springer.
- Mark Allen Weles: Data Structure & Algorithm Analysis in C Second Addition. Addison-Wesley publishing
- Thomas H. Cormen, Charles E, Leiserson& Ronald L. Rivest: Introduction to Algorithms. Prentice-Hall of India Pvt. Limited, New Delhi
- Timothy A. Budd, Classic Data Structures in C++, Addison Wesley.

Unit 02: Arrays vs Linked Lists

CONTENTS

Objectives

Introduction

2.1 Arrays

2.2 Types of Arrays

2.3 Types of Array Operations

2.4 Linked list

2.5 Types of linked list

Summary

Keywords

Self-Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Learn basic concepts of arrays
- Understand the basics of linked list
- Describe the types of array operations
- Discuss the operations of linked lists

Introduction

A data structure consists of a group of data elements bound by the same set of rules. The data elements also known as members are of different types and lengths. We can manipulate data stored in the memory with the help of data structures. The study of data structures involves examining the merging of simple structures to form composite structures and accessing definite components from composite structures. An array is an example of one such composite data structure that is derived from a primitive data structure.

An array is a set of similar data elements grouped together. Arrays can be one-dimensional or multidimensional. Arrays store the entries sequentially. Elements in an array are stored in continuous locations and are identified using the location of the first element of the array.

2.1 Arrays

An array is a data type, much like a variable as both array and variable hold information. However, unlike a variable, an array can hold several pieces of data called elements. Arrays can hold any type of data, which includes string, integers, Boolean, and so on. An array can also handle other variables as well as other arrays. An integer index identifies the individual elements of an array.

Arrays are allocated the memory in a strictly contiguous fashion. The simplest array is one-dimensional array which is a list of variables of same data type. An array of one-dimensional arrays

is called a two-dimensional array; array of two-dimensional arrays is three-dimensional array and so on.

The members of the array can be accessed using positive integer values (indicating their order in the array) called subscript or index.

200	120	-78	100	0
a[0]	a[1]	a[2]	a[3]	a[4]

The description of this array is listed below:

Name of the array : a

Data type of the array : integer

Number of elements : 5

Valid index values : 0, 1, 2, 3, 4

Value stored at the location a[0] : 200

Value stored at the location a[1] : 120

Value stored at the location a[2] : -78

Value stored at the location a[3] : 100

Value stored at the location a[4] : 0

Initializing an Array

We can initialize an array by assigning values to the elements during declaration. We can access the element by specifying its index. While initializing an array, the initial values are given sequentially separated by commas and enclosed in braces.



Example:

Consider the elements 10, 20, 30, and 40. The array can be represented as:

a[4]={10, 20, 30, 40}

The elements can be stored in an array as shown below:

a[0] = 10

a[1] = 20

a[2] = 30

a[3] = 40

The element 20 can be accessed by referencing a[1].

Now, consider n number of elements in an array. Hence, to access any element within the array, we use a[i], where i is the value between 0 to n-1.

The corresponding code used in C language to read n number of integers in an array is:

```
for(i= 0; i<n; i++)
{
scanf("%d",&a[i]);
}
```

Array Initialization in its Declaration

A variable is initialized in its declaration.



Example:

```
int value = 10;
```

Here, the value 10 is called an initializer.

Similar to a variable, we can initialize an array at the time of its declaration. The following example shows an array initialization.



Example:

```
int a[5] = {10, 11, 12, 13, 14};
```

In this declaration, a[0] is initialized to 10, a[1] is initialized to 11, and so on. There must be at least one

initial value between braces. If the number of initialized array elements is lesser than the declared size,

then the remaining array elements are assigned the value 0.

If we provide all the array elements during initialization, it is not necessary to specify the array size. The compiler automatically counts the number of elements and reserves the space in the memory for the array.



Example:

```
int a[] = {10, 20, 30, 40};
```

Here the compiler reserves four spaces for array a.

2.2 Types of Arrays

The elements in an array are referred either by a single subscript or by two or more subscripts. Hence, the arrays are of two types namely, one-dimensional array and multidimensional array, based on the subscript referred. A two-dimensional array is also a type of multidimensional array. When the array is referred by a single subscript, then it is known as one-dimensional array or linear array. When the array is referred by two subscripts, it is known as a two-dimensional array. Some programming languages allow more than two or three subscripts and these arrays are known as multidimensional arrays.

According to the number of subscripts required to access an array element, arrays can be of following types:

1. One-dimensional array
2. Multi-dimensional array

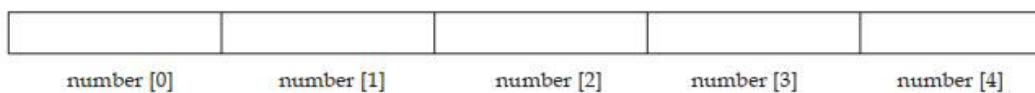
Linear Array

A linear or one-dimensional array is a structured collection of elements (often called array elements). It can be accessed individually by specifying the position of each element by an index value.

Example: If we want to store a set of five numbers by an array variable number. Then it will be accomplished in the following way:

```
int number [5];
```

This declaration will reserve five contiguous memory locations capable of storing an integer type value each, as shown below:



Now let us see how individual elements of linear array are accessed. The syntax for accessing an array component is:

ArrayName[IndexExpression]

The IndexExpression must be an integer value. The integer value can be of char, short int, long int, or

Boolean value because these are integral data types. The simplest form of index expression is a constant.



Example:

If we consider an array number[25], then,

number[0] specifies the 1st component of the array

number[1] specifies the 2nd component of the array

number[2] specifies the 3rd component of the array

number[3] specifies the 4th component of the array

number[4] specifies the 5th component of the array

.

.

.

number[23] specifies the 24th component of the array

To store and print values from the number array, we can perform the following:

```
for(int i=0; i< 25; i++)
{
    number[i]=i; // Storing a number in each array element
    printf("%d", number[i]); //Printing the value
}
```

Multidimensional Array

Multidimensional arrays are also known as "arrays of arrays." Programming languages often need to store and manipulate two or more dimensional data structures such as, matrices, tables, and so on. When programming languages use two subscripts they are known as two-dimensional arrays. One subscript denotes a row and the other denotes a column.

The declaration of two-dimension array is as follows:

```
data_type array_name[row_size][column_size];
```



Example:

```
int m[5][10]
```

Here, m is declared as a two dimensional array having 5 rows (numbered from 0 to 4) and 10 columns (numbered from 0 to 9). The first element of the array is m[0][0] and the last row last column is m[4][9]

Now let us discuss a three-dimensional array. A three-dimensional array is considered as an array of two-dimensional arrays.



Example:

A three dimensional array is created as follows:

```
int bigArray[ ][ ][ ] = new int [10][10][4];
```

This will create an array named bigArray containing 400 integers. We can access any element of this array by using 3 indices.



Example:

Suppose we want to assign a value 312 to the element at position 3 down, 7 across, and 2 in, then we write it as:

```
bigArray [2][6][1] = 312;
```

Initialization of Multidimensional Arrays

Like the one dimension arrays, two-dimensional arrays are also initialized by declaring a list of initial values enclosed in braces.



Example:

```
int table[2][3]={0,0,0,1,1,1};
```

The table array initializes the elements of first row to 0 and the second row to

1. The initialization is done row by row. The above statement can be equivalently written as:

```
int table[2][3]={{0,0,0},{1,1,1}}
```

Three or four-dimensional arrays are more complicated. They can also be initialized by declaring a list of initial values enclosed in braces.



Example:

```
int table[3][3][3]={1,2,3,4,5 6,7,8,.....27 };
```

This will create an array named table containing 27 integers. We can access any element of this array by using 3 indices.

The method to access table[1][1][1], is as shown below:

The values for array - table[3][3][3] are as follows:

{1, 2, 3}

{4, 5, 6}

{7, 8, 9}

{10, 11, 12}

{13, 14, 15}

{16, 17, 18}

{19, 20, 21}

{22, 23, 24}

{25, 26, 27}

The values in the array can be accessed using three for loops. The loop contains three variables i, j, and k respectively. This is as shown below:

```
for(i=0;i<3;i++)
{
    for(j=0;j<3;j++)
    for(k=0;k<3;k++)
    {
        printf("%d\t",table[i][j][k]);
    }
}
```

```

}
printf("\n");
}
}

printf("%d", table[1][1][1]);

```

For every iteration of the i, j and k loops, the values printed are:

```

[0][0][0] = 1
[0][0][1] = 2
[0][0][2] = 3
[1][1][1] = 14

```

2.3 Types of Array Operations

The operations performed on an array, are

1. Adding operation
2. Sorting operation
3. Searching operation
4. Traversing operation

Adding Operation

Adding elements into an array is known as insertion. The insertion of data elements is done at the end of an array. This is possible only if there is enough space in the array to add the additional elements. The elements can also be inserted in the middle of the array. Here, the average half of the array elements is moved to the next location to empty the block of memory, and to accommodate the new element.

Algorithm for Inserting an Element into an Array

Let a be an array of size N and I be the array index. Algorithm to insert an element in the MthPosition of the array a is as follows

1. Start
2. read a[N], I<-0
3. repeat for I=N to M (Decrement I by one)
4. a[I+1]<- a[I]
5. a[M]<-ELEMENT
6. M<-M+1
7. Stop

The below program illustrates the concept of inserting an element into a one-dimensional array.



Example:

```

#include<stdio.h>
#include<conio.h>

void main()
{
    int n, i, data, po_indx, a[50]; //Variable declaration

```

```

clrscr();
printf("Enter number of elements in the array\n");
/*Get the number of elements to be added to the array from the user*/
scanf("%d", &n);
printf("\nEnter %d elements\n\n", n); //Print the number of elements
for(i=0;i<n;i++) //Iterations using for loop
scanf("%d",&a[i]); // Accepting the values in the array
printf("\nEnter a data to be inserted\n");
scanf("%d",&data); //Reads the data added by user
printf("\nEnter the position of the item \n");
scanf("%d",&po_indx); //Reads the position where the data is inserted
/* Checking if the position is greater than the size of the array*/
if(po_indx-1>n)
printf("\nposition not valid\n"); //If the condition is true this will be printed
else //If the condition is false the 'else' part will get executed
{
for(i=n;i>=po_indx;i--) //Iterations using for loop
a[i]=a[i-1]; //Value of a[i-1] is assigned to a[i]
/*Value of data will be assigned to [po_indx-1] position*/
a[po_indx-1]=data;
n=n+1; //Incrementing the value of n
printf("\nArray after insertion\n"); //Print the array list after insertion
for(i=0;i<n;i++) //Use for loop and
printf("%d\t",a[i]); //Print the final array after insertion
}
getch(); //Display characters on screen
}

```

Output:

Enter number of elements in the array

5

Enter 5 elements

15 20 32 45 62

Enter a data to be inserted

77

Enter the position of the item

2

Array after insertion

15 77 20 32 45 62

In this example:

1. First, the header files are included using #include directive.

2. Then, the index, array, and the variables are declared.
3. The program accepts the number of elements in the array.
4. Using a for loop, the values are accepted and stored in the array.
5. Then, the program accepts the data along with the position where it needs to be inserted.
6. If the position to be inserted is greater than the number of elements ($po_indx-1 > n$) then the program displays "position is not valid". Otherwise, the program by means of a for loop, checks whether $i \geq po_indx$ is true and assigns the $a[i-1]$ value to $a[i]$.
7. Then data is assigned to $a[po_indx-1]$.
8. Then, the program increments the number of elements and prints the array after insertion.
9. getch() prompts the user to press a key and the program terminates.

Sorting Operation

Sorting operation arranges the elements of a list in a certain order. Efficient sorting is important for optimizing the use of other algorithms that require sorted lists to work correctly.

Sorting an array efficiently is quite complicated. There are different sorting algorithms to perform the task of sorting, but here we will discuss only Bubble Sort.

Bubble Sort

Bubble sort is a simple sorting technique when compared to other sorting techniques. The bubble sort algorithm starts from the very first element of the data set. In order to sort elements in the ascending order, the algorithm compares the first two elements of the data set. If the first element is greater than the second, then the numbers are swapped.

This process is carried out for each pair of adjacent elements to the end of the data set until no swaps occur on the last pass. This algorithm's average and worst case performance is $O(2n)$ as it is rarely used to sort large, unordered data sets.

Bubble sort can always be used to sort a small number of items where efficiency is not a high priority. Bubble sort may also be effectively used to sort a partially sorted list.

Algorithm for Sorting an Array

Let A be an array containing data with N elements. This algorithm sorts the elements in A as follows:

1. Start
2. Repeat Steps 3 and 4 for $K = 1$ to $N-1$
3. Set $PTR := 1$ [Initializes pass pointer PTR]
4. Repeat while $PTR \leq N - K$: [Executes pass]
 - If $A[PTR] > A[PTR+1]$, then:
 - Interchange $A[PTR]$ and $A[PTR + 1]$
 - [End of If structure]
 - Set $PTR := PTR + 1$
 - [End of inner loop]
- [End of Step 2 outer loop]
5. Exit

In the algorithm, there is an inner loop, which is controlled by the variable PTR, and an index K controls the outer loop. K is used as a counter and PTR is used as an index.

The below program illustrates the concept of sorting an array using bubble sort.



Example:

```
#include <stdio.h>
#include <conio.h>

int A[8] = {55, 22, 2, 43, 12, 8, 32, 15}; //Declaring the array with 8 elements
int N = 8; //Size of the array
void BUBBLE (void); //BUBBLE Function declaration

void main()
{
    int i; //Variable declaration
    clrscr();
    /*Printing the values in the array*/
    printf("\n\nValues present in array A =");
    for (i=0; i<8; i++) //Iterations using for loop
        printf(" %d, ", A[i]); //Printing the array
    BUBBLE(); //BUBBLE function is called
    /*Printing the values from the array after sorting*/
    printf("\n\nValues present in the array after sorting =");
    for (i=0; i<8; i++) //Iterations
        printf(" %d, ", A[i]); // Printing the array after sorting
    getch(); // waits for a key to be pressed
}

void BUBBLE(void) //BUBBLE Function definition
{
    int K, PTR, TEMP; //Declaration variables
    for(K=0; K <= (N-2); K++) //Iterations
    {
        PTR = 0; // Assign 0 to variable PTR
        while(PTR <= (N-K-1-1)) //Checking if PTR <= (N-K-1-1)
        {
            /* Checking if the element at A[PTR] is greater than A[PTR+1]*/
            if(A[PTR] > A[PTR+1])
            {
                TEMP = A[PTR];
                A[PTR] = A[PTR+1];
                A[ PTR +1] = TEMP;
            }
            /*Increment the array index*/
            PTR = PTR+1;
        }
    }
}
```

}

Output:

Values present in A[8] = 55, 22, 2, 43, 12, 8, 32, 15

Values present in A[8] after sorting = 2, 8, 12, 15, 22, 32, 43, 55

In this example:

1. First, the header files are included using #include directive.
2. Then, the array A is declared globally along with the array elements and the size.
3. Then, inside the main function the variable i is declared.
4. The values in the array are printed using a for loop.
5. Next, the Bubble function is called. The sorting operation is carried out and values present in the array are printed.
6. getch() prompts the user to press a key. Then the program terminates.
7. In The BUBBLE function the variables K, PTR and TEMP are declared as integers.
8. PTR is set to 0.
9. Within the while loop the adjacent array elements are compared. If the element at a lower position is greater than the element at the next position, both the elements are interchanged.
10. The array index is then incremented.

Searching Operation

Searching is an operation used for finding an item with specified properties among a collection of items. In a database, the items are stored individually as records, or as elements of a search space addressed by a mathematical formula or procedure. The mathematical formula or procedure may be the root of an equation containing integer variables.

Search operation is closely related to the concept of dictionaries. Dictionaries are a type of data structure that support operations such as, search, insert, and delete.

Computer systems are used to store large amounts of data. From these large amount of data, individual records are retrieved based on some search criterion. The efficient storage of data is an important issue to facilitate fast searching.

There are many different searching techniques or algorithms. The selection of algorithm depends on the way the information is organized in memory. Now, we will discuss linear searching technique.

Algorithm for Linear search

Let A be a linear array with N elements and ITEM be the given item of information. The search algorithm will find the location LOC of ITEM in A or sets LOC :=0 if the search fails. The algorithm is as follows:

1. Start
2. [Insert ITEM at the end of A.] Set A[N+1] :=ITEM
3. [Initialize counter.] Set LOC :=1
4. [Search for ITEM.]
 - (a) Repeat while A[LOC] ≠ ITEM:
 - (b) Set LOC := LOC + 1
- [End of loop]
5. [Successful?] If LOC = N + 1, then: Set LOC := 0

6. Exit

Traversing Operation

Traversing an array refers to moving in inward and outward direction to access each element in an array. To traverse an array, one can use for loop. The array elements are accessed using an array index or a pointer of type similar to that of array elements. To access the elements using a pointer, the pointer must be initialized with the base address of the array. Traversing operation also involves printing the elements in an array.

Algorithm for Traversal Operation

Let X be an array of size N. You need to traverse through the array and perform the required operations on each element of the array. Let the required operation be OP. Here, i is the array index and the lower bound starts with 0. The algorithm for traversing a given array is as follows:

1. Start
2. read X[N], i=0
3. repeat for I = 0, 1, 2.....N

OP on X[i]

4. Stop



Example:

```
#include<stdio.h>
#include<conio.h>
#define SIZE 20 //Define array size
void main()
{
float sum(float[], int); //Function declaration
float x[SIZE], Sum_total=0.0;
int i, n; //Variable declaration
clrscr();
printf("Enter the number of elements in array\n");
scanf(" %d", &n); //Reads the data added by user
printf("Enter %d elements:\n", n); //Printing the values in the array
for(i=0; i<n; i++) //Iterations using for loop
/* Input the elements of the array (Traverse operation)*/
scanf(" %f", &x[i]);
printf("The elements of array are:\n\n"); //Printing the elements of the array
for(i=0; i<n; i++) //Iterations using for loop
/*print the elements of array in floating point form(Traverse operation)*/
printf(" %.2f\t", x[i]);
/*Call the function sum and store the value returned in Sum_total*/
Sum_total = sum(x, n);
/*Printing the sum*/
printf("\n\nSum of the given array is: %.2f\n", Sum_total);
getch(); //wait until a key is pressed
}
```

```
float sum(float x[], int n) //Function declaration
{
    int i; //Variable declaration
    float total=0.0; //the variable total is set to 0.0
    for(i=0; i<n; i++) //Iterations
        total+=x[i]; //each element x[i] is added to the value of total
    return(total); //Returning the total value
}
```

Output:

Enter the number of elements in array

5

Enter 5 elements

14 15 16 17 18

The elements of array are:

14.00 15.00 16.00 17.00 18.00

Sum of the given array is: 80.00

In this example:

1. First, the header files are included using #include directive.
2. Using the #define directive, the array size, SIZE, is set to 20.
3. In the main() function, the function sum and the variables are declared.
4. A for loop is used accept the elements of the array.
5. The next for loop prints the elements of the array.
6. The program calls the sum() function to add all the elements of the array. The value returned by the sum() function is stored in the variable Sum_total.
7. The program then prints the sum of the elements of the array.
8. getch() prompts the user to press a key to exit the program.
9. The function sum that accepts two arguments and returns a float value is defined. The function sum does the following steps:
 - (a) Initializes an integer variable i.
 - (b) Initializes a float variable total and assigns 0.0 to it.
 - (c) Adds the elements of the array using a for loop and the result is stored in total.
 - (d) Finally, it returns the value of total.

2.4 Linked list

Linked lists are the most common data structures. They are referred to as an array of connected objects where data is stored in the pointer fields. Linked lists are useful when the number of elements to be stored in a list is indefinite.

Concept of Linked Lists

An array is represented in memory using sequential mapping, which has the property that elements are fixed distance apart. But this has the following disadvantage. It makes insertion or

deletion at any arbitrary position in an array a costly operation, because this involves the movement of some of the existing elements.

When we want to represent several lists by using arrays of varying size, either we have to represent each list using a separate array of maximum size or we have to represent each of the lists using one single array. The first one will lead to wastage of storage, and the second will involve a lot of data movement.

So we have to use an alternative representation to overcome these disadvantages. One alternative is a linked representation. In a linked representation, it is not necessary that the elements be at a fixed distance apart. Instead, we can place elements anywhere in memory, but to make it a part of the same list, an element is required to be linked with a previous element of the list. This can be done by storing the address of the next element in the previous element itself. This requires that every element be capable of holding the data as well as the address of the next element. Thus every element must be a structure with a minimum of two fields, one for holding the data value, which we call a data field, and the other for holding the address of the next element, which we call link field.

Therefore, a linked list is a list of elements in which the elements of the list can be placed anywhere in memory, and these elements are linked with each other using an explicit link field, that is, by storing the address of the next element in the link field of the previous element.

This program uses a strategy of inserting a node in an existing list to get the list created. An insert function is used for this. The insert function takes a pointer to an existing list as the first parameter, and a data value with which the new node is to be created as a second parameter, creates a new node by using the data value, appends it to the end of the list, and returns a pointer to the first node of the list. Initially the list is empty, so the pointer to the starting node is NULL.

Therefore, when insert is called first time, the new node created by the insert becomes the start node. Subsequently, the insert traverses the list to get the pointer to the last node of the existing list, and puts the address of the newly created node in the link field of the last node, thereby appending the new node to the existing list. The main function reads the value of the number of nodes in the list. Calls iterate that many times by going in a while loop to create the links with the specified number of nodes.

2.5 Types of linked list

Single linked list

Double linked list

Circular linked list

A doubly-linked list is a linked data structure that consists of a set of sequentially linked records called nodes. Each node contains two fields, called links, that are references to the previous and to the next node in the sequence of nodes.

In the single linked list each node provides information about where the next node is in the list. It faces difficulty if we are pointing to a specific node, then we can move only in the direction of the links. It has no idea about where the previous node lies in memory. The only way to find the node which precedes that specific node is to start back at the beginning of the list. The same problem arises when one wishes to delete an arbitrary node from a single linked list. Since in order to easily delete an arbitrary node one must know the preceding node. This problem can be avoided by using Doubly Linked List, we can store in each node not only the address of next node but also the address of the previous node in the linked list. A node in Doubly Linked List has three fields

1. Data
2. Previous Link
3. Next Link

Prev. Link	Data field	Next Link
------------	------------	-----------

Implementation of Doubly Linked List

Structure of a node of Doubly Linked List can be defined as:

```
struct node
{
    int data;
    struct node *llink;
    struct node *rlink;
}
```

Circular Linked List

Circular Linked List is another remedy for the drawbacks of the Single Linked List besides Doubly Linked List. A slight change to the structure of a linear list is made to convert it to circular linked list; link field in the last node contains a pointer back to the first node rather than a Null.

Representation of Linked List

Because each node of an element contains two parts, we have to represent each node through a structure.

While defining linked list we must have recursive definitions:

```
struct node
{
    int data;
    struct node * link;
}
```

Here, link is a pointer of struct node type i.e. it can hold the address of variable of struct node type. Pointers permit the referencing of structures in a uniform way, regardless of the organization of the structure being referenced. Pointers are capable of representing a much more complex relationship between elements of a structure than a linear order.

Initialization:

```
main()
{
    struct node *p, *list, *temp;
    list = p = temp = NULL;
    .
    .
    .
}
```



Example:

Program:

```
# include <stdio.h>
# include <stdlib.h>

struct node
{
    int data;
    struct node *link;
```

```

};
struct node *insert(struct node *p, int n)
{
    struct node *temp;
    /* if the existing list is empty then insert a new node as the
    starting node */
    if(p==NULL)
    {
        p=(struct node *)malloc(sizeof(struct node)); /* creates new
        node data value passes
        as parameter */
        if(p==NULL)
        {
            printf("Error\n");
            exit(0);
        }
        p-> data = n;
        p-> link = p; /* makes the pointer pointing to itself because
        it is a circular list*/
    }
    else
    {
        temp = p;
        /* traverses the existing list to get the pointer to the last node
        of it */
        while (temp->link != p)
        temp = temp->link;
        temp-> link = (struct node *)malloc(sizeof(struct node)); /*
        creates new node using
        data value passes as
        parameter and puts its
        address in the link field
        of last node of the
        existing list*/
        if(temp -> link == NULL)
        {
            printf("Error\n");
            exit(0);
        }
        temp = temp->link;
    }
}

```

```

temp-> data = n;
temp-> link = p;
}
return (p);
}
void printlist( struct node *p )
{
struct node *temp;
temp = p;
printf("The data values in the list are\n");
if(p!= NULL)
{
do
{
printf("%d\t",temp->data);
temp=temp->link;
} while (temp!= p);
}
else
printf("The list is empty\n");
}
void main()
{
int n;
int x;
struct node *start = NULL ;
printf("Enter the nodes to be created \n");
scanf("%d",&n);
while ( n -- > 0 )
{
printf( "Enter the data values to be placed in a node\n");
scanf("%d",&x);
start = insert ( start, x);
}
printf("The created list is\n");
printlist( start );
}

```

Deleting the Specified Node in Singly Linked List

To delete a node, first we determine the node number to be deleted (this is based on the assumption that the nodes of the list are numbered serially from 1 to n). The list is then traversed to get a pointer to the node whose number is given, as well as a pointer to a node that appears before the

node to be deleted. Then the link field of the node that appears before the node to be deleted is made to point to the node that appears after the node to be deleted, and the node to be deleted is freed.



Lab Exercise:

```
# include <stdio.h>
# include <stdlib.h>

struct node *delet( struct node *, int );
int length ( struct node * );

struct node
{
    int data;
    struct node *link;
};

struct node *insert(struct node *p, int n)
{
    struct node *temp;
    if(p==NULL)
    {
        p=(struct node *)malloc(sizeof(struct node));
        if(p==NULL)
        {
            printf("Error\n");
            exit(0);
        }
        p-> data = n;
        p-> link = NULL;
    }
    else
    {
        temp = p;
        while (temp->link != NULL)
            temp = temp->link;
        temp-> link = (struct node *)malloc(sizeof(struct node));
        if(temp -> link == NULL)
        {
            printf("Error\n");
            exit(0);
        }
        temp = temp->link;
        temp-> data = n;
        temp-> link = NULL;
    }
    return (p);
}
```

```

}
void printlist( struct node *p )
{
printf("The data values in the list are\n");
while (p!= NULL)
{
printf("%d\t",p-> data);
p = p->link;
}
}
void main()
{
int n;
int x;
struct node *start = NULL;
printf("Enter the nodes to be created \n");
scanf("%d",&n);
while ( n- > 0 )
{
printf( "Enter the data values to be placed in a node\n");
scanf("%d",&x);
start = insert ( start, x );
}
printf(" The list before deletion id\n");
printlist( start );
printf("% \n Enter the node no \n");
scanf( " %d",&n);
start = delet (start , n );
printf(" The list after deletion is\n");
printlist( start );
}
/* a function to delete the specified node*/
struct node *delet( struct node *p, int node_no )
{
struct node *prev, *curr ;
int i;
if (p == NULL )
{
printf("There is no node to be deleted \n");
}
}

```

```

else
{
if ( node_no> length (p))
{
printf("Error\n");
}
else
{
prev = NULL;
curr = p;
i = 1 ;
while ( i<node_no )
{
prev = curr;
curr = curr->link;
i = i+1;
}
if ( prev == NULL )
{
p = curr ->link;
free ( curr );
}
else
{
prev -> link = curr ->link ;
free ( curr );
}
}
return(p);
}

/* a function to compute the length of a linked list */
int length ( struct node *p )
{
int count = 0 ;
while ( p != NULL )
{
count++;
p = p->link;
}
}

```

```
return ( count ) ;  
}
```

Inserting a Node after the Specified Node in a Singly Linked List

To insert a new node after the specified node, first we get the number of the node in an existing list after which the new node is to be inserted. This is based on the assumption that the nodes of the list are numbered serially from 1 to n. The list is then traversed to get a pointer to the node, whose number is given. If this pointer is x, then the link field of the new node is made to point to the node pointed to by x, and the link field of the node pointed to by x is made to point to the new node. Figures 2.3 and 2.4 show the list before and after the insertion of the node, respectively.

Insertion in Linked List can happen at following places:

At the beginning of the linked list.

At the end of the linked list.

At a given position in the linked list.

Algorithm: Insertion at beginning

Step 1: IF PTR = NULL

Write OVERFLOW

Go to Step 7

[END OF IF]

Step 2: SET NEW_NODE = PTR

Step 3: SET PTR = PTR → NEXT

Step 4: SET NEW_NODE → DATA = VAL

Step 5: SET NEW_NODE → NEXT = HEAD

Step 6: SET HEAD = NEW_NODE

Step 7: EXIT

Deletion from a Linked List

Delete from beginning

Delete from end

Delete from middle/ given position

Find the previous node of the node to be deleted.

Change the next pointer of the previous node

Free the memory of the deleted node.

In case of first node deletion, we need to update the head of the linked list.

Algorithm: Deletion at beginning

Step 1: IF HEAD = NULL

Write UNDERFLOW

Go to Step 5

[END OF IF]

Step 2: SET PTR = HEAD

Step 3: SET HEAD = HEAD → NEXT

Step 4: FREE PTR

Step 5: EXIT

Searching in linked list

Searching is performed to find the location of a particular element in the list. Traversing is performed in the list and make the comparison of every element of the list with the specified element. If the element is matched with any of the list element then the location of the element is returned from the function.

Algorithm: Searching in linked list

Step 1: SET PTR = HEAD

Step 2: Set I = 0

STEP 3: IF PTR = NULL

WRITE "EMPTY LIST"

GOTO STEP 8

END OF IF

STEP 4: REPEAT STEP 5 TO 7 UNTIL PTR != NULL

STEP 5: if ptr → data = item

write i+1

End of IF

STEP 6: I = I + 1

STEP 7: PTR = PTR → NEXT

[END OF LOOP]

STEP 8: EXIT

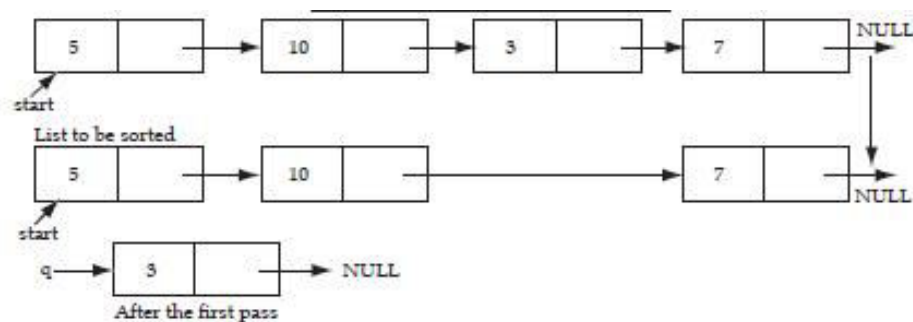
Linked List Common Errors

Here is summary of common errors of linked lists. Read these carefully, and read them again when you have problem that you need to solve.

1. Allocating a new node to step through the linked list; only a pointer variable is needed.
2. Confusing the and the -> operators.
3. Not setting the pointer from the last node to 0 (null).
4. Not considering special cases of inserting/removing at the beginning or the end of the linked list.
5. Applying the delete operator to a node (calling the operator on a pointer to the node) before it is removed. Delete should be done after all pointer manipulations are completed.
6. Pointer manipulations that are out of order. These can ruin the structure of the linked list.

Sorting and Reversing a Linked List

To sort a linked list, first we traverse the list searching for the node with a minimum data value. Then we remove that node and append it to another list which is initially empty. We repeat this process with the remaining list until the list becomes empty, and at the end, we return a pointer to the beginning of the list to which all the nodes are moved.



Sorting of linked list

To reverse a list, we maintain a pointer each to the previous and the next node, then we make the link field of the current node point to the previous, make the previous equal to the current, and the current equal to the next.

Arrays vs. Linked list

Array	Linked list
Data elements are stored in contiguous locations in memory.	New elements can be stored anywhere and a reference is created for the new element using pointers.
Insertion and Deletion operations are costlier since the memory locations are consecutive and fixed.	Insertion and Deletion operations are fast and easy in a linked list.
Memory is allocated during the compile time (<i>Static memory allocation</i>).	Memory is allocated during the run-time (<i>Dynamic memory allocation</i>).
Size of the array must be specified at the time of array declaration/initialization.	Size of a Linked list grows/shrinks as and when new elements are inserted/deleted.

Summary

- An array is a set of same data elements grouped together. Arrays can be one-dimensional or multidimensional.
- A linear or one-dimensional array is a structured collection of elements (often called as array elements) that are accessed individually by specifying the position of each element with a single index value.
- Multidimensional arrays are nothing but "arrays of arrays". Two subscripts are used to refer to the elements.
- The operations that are performed on an array are adding, sorting, searching, and traversing.
- Traversing an array refers to moving in inward and outward direction to access each element in an array.
- Linked list is a technique of dynamically implementing a list using pointers. A linked list contains two fields namely, data field and link field.
- A singly-linked list consists of only one pointer to point to another node and the last node always points to NULL to indicate the end of the list.
- A doubly-linked list consists of two pointers, one to point to the next node and the other to point to the previous node.
- In a circular singly-linked list, the last node always points to the first node to indicate the circular nature of the list.
- A circular doubly-linked list consists of two pointers for forward and backward traversal and the last node points to the first node.

- Searching operation involves searching for a specific element in the list using an associated key.
- Insertion operation involves inserting a node at the beginning or end of a list.
- Deletion operation involves deleting a node at the beginning or following a given node or at the end of a list.

Keywords

Non-linear Data Structure: Every data item is attached to several other data items in a way that is specific for reflecting relationships. The data items are not arranged in a sequential structure.

Searching: Finding the location of the record with a given key value, or finding the locations of all records, which satisfy one or more conditions.

Traversing: Accessing each record exactly once so that certain items in the record may be processed.

Circular Linked List: A linear linked list in which the last element points to the first element, thus, forming a circle.

Doubly Linked List: A linear linked list in which each element is connected to the two nearest elements through pointers.

Self-Assessment

1. Which one is correct statement?
 - A. Search in array is delete an element from array
 - B. Search in array is find an element from array
 - C. Search in array is insert an element in array
 - D. None of above
2. Which is not correct syntax?
 - A. `abcname[ ];`
 - B. `abcname[10];`
 - C. `abcname[3] = {20,25,35};`
 - D. `abcname[5] = {15;20;28};`
3. Searching is a process in which we find element in array
 - A. True
 - B. False
4. Operations can be performed on array
 - A. Sorting
 - B. Merging
 - C. Traversing
 - D. All of above
5. To merge two arrays, how many variables are required (minimum)?

- A. 1
- B. 2
- C. 5
- D. 3

6. Elements of arrays are stored in memory locations

- A. Random
- B. Sequential
- C. Both random and sequential
- D. None of above

7. Which is correct statement?

- A. Insertion at the given index of an array
- B. Insertion after the given index of an array
- C. Insertion before the given index of an array
- D. All of above

8. Traversal is process of visit each element of an array

- A. True
- B. False

9. Which statement is incorrect?

- 1. `int arr[MAX]= {10,12,15,20},i,val;`
- 2. `printf("the array element are \n");`
- 3. `for(i=0;i<4;i++)`
- 4. none of above

- A. 1
- B. 2
- C. 3
- D. 4

10. Array is____

- A. A group of elements of same data type
- B. Array elements are stored in memory in continuous or contiguous locations
- C. An array contains more than one element
- D. All of above

11. What are the advantages of arrays?

- A. Objects of mixed data types can be stored
- B. Easier to store elements of same data type
- C. Elements in an array cannot be sorted

D. Index of first element of an array is 1

12. The index of the first element in an array is_____

- A. 1
- B. 2
- C. -1
- D. 0

13. Linked list consist of...

- A. Data field
- B. Link field
- C. Both data and link field
- D. None of above

14. What are the shortcomings of array?

- A. Memory allocation
- B. Memory efficiency
- C. Execution time
- D. All of above

15. What are the types of linked list?

- A. Single
- B. Double
- C. Circular
- D. All of above

16. Operations performed on Linked list are...

- A. Insertion
- B. Deletion
- C. Search
- D. All of above

17. Insertion in Linked List can happen at following places

- A. At the beginning of the linked list.
- B. At the end of the linked list.
- C. At a given position in the linked list.
- D. All of above

18. Linked list is considered as an example of _____ type of memory allocation

- A. Static
- B. Heap
- C. Dynamic

D. Compile time

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. B | 2. D | 3. A | 4. D | 5. D |
| 6. B | 7. D | 8. A | 9. B | 10. D |
| 11. B | 12. D | 13. C | 14. D | 15. D |
| 16. D | 17. D | 18. C | | |

Review Questions

1. Define array and its types.
2. Give an example of multidimensional array.
3. Discuss any two types of array initialization methods with example.
4. Discuss different sorting methods.
5. Write a program to sort the elements of a linked list.
6. Differentiate between array and linked list with suitable example.
7. Discuss different operation performed with linked list.
8. Discuss advantages of linked list as compared to arrays.



Further Readings

Data Structures and Efficient Algorithms, Burkhard Monien, Thomas Ottmann, Springer.

Kruse Data Structure & Program Design, Prentice Hall of India, New Delhi

Mark Allen Weles: Data Structure & Algorithm Analysis in C Second Addition. Addison-Wesley publishing

Sorenson and Tremblay: An Introduction to Data Structure with Algorithms.

Thomas H. Cormen, Charles E. Leiserson & Ronald L. Rivest: Introduction to Algorithms. Prentice-Hall of India Pvt. Limited, New Delhi

Timothy A. Budd, Classic Data Structures in C++, Addison Wesley.



Web Links

www.en.wikipedia.org

www.web-source.net

www.webopedia.com

<https://www.geeksforgeeks.org/data-structures/>

<https://www.programiz.com/dsa/data-structure-types>

Unit 03: Stacks

CONTENTS

Objectives

Introduction

3.1 Stack Structure

3.2 Basic Operations of Stack

3.3 Implementation of Stacks

3.4 Applications of Stacks

Summary

Keywords

Self Assessment

Answers for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Learn fundamentals of stacks
- Explain the basic operations of stack
- Explain the implementation and applications of stacks

Introduction

Stacks are simple data structures and an important tool in programming language. Stacks are linear lists which have restrictions on the insertion and deletion operations. These are special cases of ordered list in which insertion and deletion is done only at the ends.

The basic operations performed on stack are push and pop. Stack implementation can be done in two ways - static implementation or dynamic implementation. Stack can be represented in the memory using a one-dimensional array or a singly linked list.

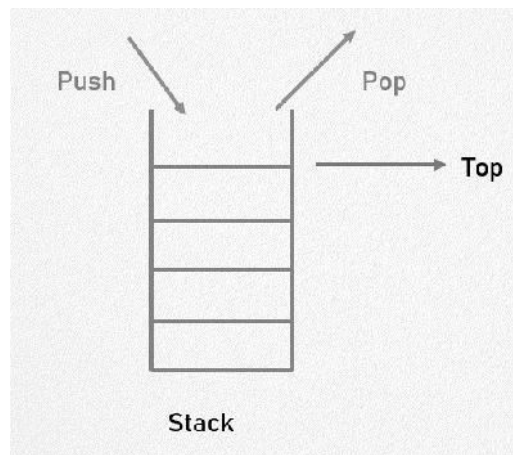
Stack is another linear data structure having a very interesting property. Unlike arrays and link lists, an element can be inserted and deleted not at any arbitrary position but only at one end. Thus, one end of a stack is sealed for insertion and deletion while the other end allows both the operations.

3.1 Stack Structure

The stack data structure is used to maintain records of a file in which the order among the records of file is not important. Figure 7.1 displays the structure of a stack where stack is like a hollow cylinder with a closed bottom end and an open top end. In the stack data structure, the records are added and deleted at the top end. Last-In-First-Out (LIFO) principle is followed to retrieve records from the stack. The records added last are accessed first.

A stack is a linear data structure in as much as its member elements are ordered as 1st, 2nd,.... and last. However, an element can be inserted in and deleted from only one end. The other end remains sealed. This open end to which elements can be inserted and deleted from is called stack top or top of the stack. Consequently, the elements are removed from a stack in the reverse order of insertion.

A stack is said to possess LIFO (Last In First Out) property. A data structure has LIFO property if the element that can be retrieved first is the one that was inserted last.



3.2 Basic Operations of Stack

The basic operations of stack are to:

1. Insert an element in the stack (Push operation)
2. Delete an element from the stack (Pop operation)

Push Operation

The procedure to insert a new element to the stack is called push operation. The push operation adds an element on the top of the stack. 'Top' refers to the element on the top of stack. Push makes the 'Top' point to the recently added element on the stack. After every push operation, the 'Top' is incremented by one. When the array is full, the status of stack is FULL and the condition is called stack overflow. No element can be inserted when the stack is full

Algorithm to Implement Push Operation on Stack

```
PUSH (STACK, n, top, item) /* n = size of stack*/  
if (top = n) then STACK_FULL; /* checks for stack overflow */  
else  
{ top = top+1; /* increases the top by 1 */  
STACK [top] = item ;} /* inserts item in the new top position */  
end PUSH
```

Pop Operation

The procedure to delete an element from the top of the stack is called pop operation. After every pop operation, the 'Top' is decremented by one. When there is no element in the stack, the status of the stack is called empty stack or stack underflow. The pop operation cannot be performed when it is in stack underflow condition.

Algorithm to Implement Pop Operation in a Stack

```
POP (STACK, top, item)  
if (top = 0) then STACK_EMPTY; /* check for stack underflow*/  
else { item = STACK [top]; /* remove top element*/  
top = top - 1; /* decrement stack top*/  
}  
end POP
```

3.3 Implementation of Stacks

There are two basic methods for the implementation of stacks – one where the memory is used statically and the other where the memory is used dynamically.

Array-based Implementation

A stack is a sequence of data elements. To implement a stack structure, an array can be used as it is a storage structure. Each element of the stack occupies one array element. Static implementation of stack can be achieved using arrays. The size of the array, once declared, cannot be changed during the program execution. Memory is allocated according to the array size. The memory requirement is determined before the compilation. The compiler provides the required memory. This is suitable when the exact number of elements is known. The static allocation is an inefficient memory allocation technique because if fewer elements are stored than declared, the memory is wasted and if more elements need to be stored than declared, the array cannot expand. In both the cases, there is inefficient use of memory.

The following pseudo-code shows the array-based implementation of a stack. In this, the elements of the stack are of type T.

```
struct stk
{
    T array[max_size];
    /* max_size is the maximum size */
    int top = -1;
    /* stack top initially given value -1 */
} stack;

void push(T e)
/*inserts an element e into the stack s*/
{
    if (stack.top == max_size)
        printf("Stack is full-insertion not possible");
    else
    {
        stack.top = stack.top + 1;
        stack.array[stack.top] = e;
    }
}

T pop()
/*Returns the top element from the stack */
{
    T x;
    if(stack.top == -1)
        printf("Stack is empty");
    else
    {
        x = stack.array[stack.top];
        stack.top = stack.top - 1;
        return(x);
    }
}
```

```

}
boolean empty()
/* checks if the stack is empty */
{
    boolean empty = false;
    if(stack.top == -1)
        empty = true else empty = false;
    return(empty);
}

void initialise()
/* This procedure initializes the stack s */
{
    stack.top = -1;
}

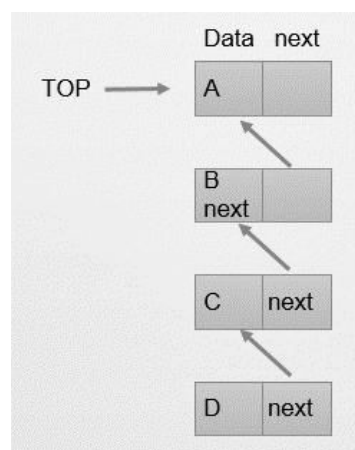
```

The above implementation strategy is easy and fast since it does not have run-time overheads. At the same time it is not flexible since it cannot handle a situation when the number of elements exceeds max_size. Also, let us say, if max_size is derided statically to 100 and a stack actually has only 10 elements, then memory space for the rest of the 90 elements would be wasted.

Linked List Representation of Stacks

The array representation of stacks is easy and convenient. However, it allows the representation of only fixed sized stacks. The size of the stack varies during program application for different applications. Representing stack using linked list can solve this problem. A singly linked list can be used to represent any stack. In a singly linked list, the data field represents the ITEM and the LINK field points to the next item.

In linked list implementation of the stack, we need to create nodes and the nodes are maintained non-contiguously in the memory. Each node contains a pointer to its immediate successor node in the stack. Stack is said to be overflown if the space left in the memory heap is not enough to create a node.



Here the memory is used dynamically. For every push operation, the memory space for one element is allocated at run-time and the element is inserted into the stack. For every pop operation, the memory space for the deleted element is de-allocated and returned to the free space pool. Hence the shortcomings of the array-based implementation are overcome. But since, this allocates memory dynamically, the execution is slowed down.

The following pseudo-code is for the pointer-based implementation of a stack. Each element of the stack is of type T.

```
struct stk
{
    T element;
    struct stk *next;
};

struct stk *stack;

void push(struct stk *p, T e)
{
    struct stk *x;
    x = new(stk);
    x->element = e;
    x->next = NULL;
    p = x;
}
```

Here the stack full condition is checked by the call to new which would give an error if no memory space could be allocated.

```
T pop(struct stk *p)
{
    struct stk *x;
    if (p == NULL)
        printf("Stack is empty");
    else
    {
        x = p;
        x = x->next;
        return(x->element);
    }
}

booleanempty(struct stk *p)
{
    if (p == NULL)
        return(true);
    else
        return(false);
}

void initialize(struct stk *p)
{
    p = NULL;
}
```

3.4 Applications of Stacks

There are numerous applications of the stack data structure in computer algorithms. It is used to store return information in the case of function/procedure/subroutine calls. Hence, one would find a stack in architecture of any Central Processing Unit (CPU). In this section, we would just illustrate a few of them.

Expression Evaluation and Conversion

Parenthesis Checking

Backtracking

Function Call

String Reversal

Memory Management

Syntax Parsing

Parenthesis checker

Parenthesis checker is used for balanced Brackets in an expression. The balanced parenthesis means that when the opening parenthesis is equal to the closing parenthesis, then it is a balanced parenthesis.

$(a+b*(c/d))$

$[10+20*(6+7)]$

$(x+y)/(c-d)$

Balanced parenthesis

$A = (50+25)$

In the above expression there is one opening and one closing parenthesis means that both opening and closing brackets are equal; therefore, the above expression is a balanced parenthesis.

Unbalanced parenthesis

$A = [(15+25)$

The above expression has two opening brackets and one closing bracket, which means that both opening and closing brackets are not equal; therefore, the above expression is unbalanced.

Algorithm

- Initialize a character stack.
- Now traverse the expression string exp.
- If the current character is a starting bracket ('(' or '{' or '[') then push it to stack.
- If the current character is a closing bracket (')' or '}' or ']') then pop from stack and if the popped character is the matching starting bracket then balanced else brackets are not balanced.
- After complete traversal, if there is some starting bracket left in stack then not balanced

Expression conversion and evaluation

Arithmetic expressions can be represented in 3 forms:

Infix notation

Postfix notation (Reverse Polish Notation)

Prefix notation (Polish Notation)

Infix Notation

Infix Notation can be represented as:

operand1 operator operand1



Example: 15 + 26

a + b

Postfix Notation

Postfix Notation can be represented as

operand1 operand2 operator



Example: 15 29 +

a b +

Prefix notation

Prefix notation can be represented as

operator operand1 operand2



Example: + 10 20

+ a b

Infix notation is used most frequently in our day to day tasks. Machines find infix notations tougher to process than prefix/postfix notations. Hence, compilers convert infix notations to prefix/postfix before the expression is evaluated.

The precedence of operators needs to be taken care as per hierarchy

(^) > (*) > (/) > (+) > (-)

Brackets have the highest priority.

To evaluate an infix expression, We need to perform 2 steps:

- Convert infix to postfix
- Evaluate postfix

Sorting

A Sorting process is used to rearrange a given array or elements based upon selected algorithm/sort function.



Quick Sort is used for sorting a list of data elements. Quicksort is a sorting algorithm based on the divide and conquer approach. An array is divided into subarrays by selecting a pivot element. During array dividing, the pivot element should be positioned in such a way that elements less than pivot are kept on the left side and elements greater than pivot are on the right side of the pivot. The left and right subarrays are also divided using the same approach. This process continues until each subarray contains a single element

There are many different versions of quick Sort that pick pivot in different ways.

Always pick first element as pivot.

Always pick last element as pivot

Pick a random element as pivot.

Pick median as pivot.

Algorithm

```
quickSort(arr, beg, end)
    if (beg < end)
        pivotIndex = partition(arr, beg, end)
        quickSort(arr, beg, pivotIndex)
        quickSort(arr, pivotIndex + 1, end)
    partition(arr, beg, end)
    set end as pivotIndex
    pIndex = beg - 1
    for i = beg to end-1
        if arr[i] < pivot
            swap arr[i] and arr[pIndex]
    pIndex++
    swap pivot and arr[pIndex+1]
    return pIndex + 1
```

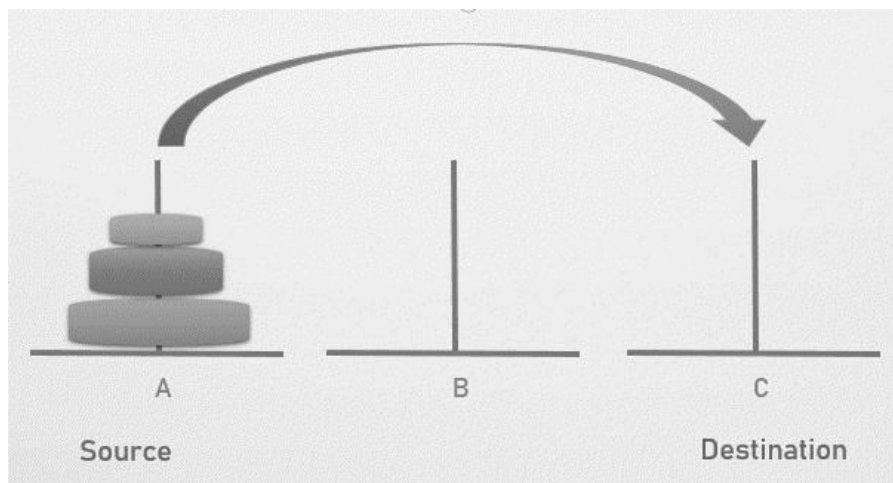
Tower of Hanoi

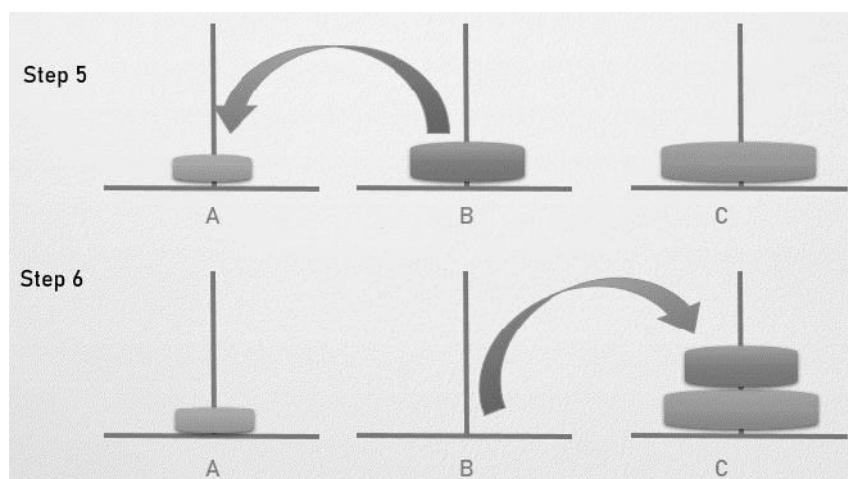
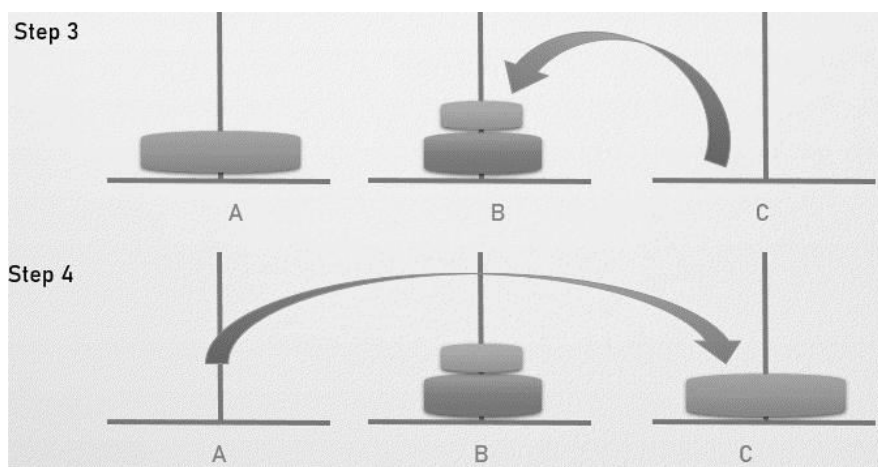
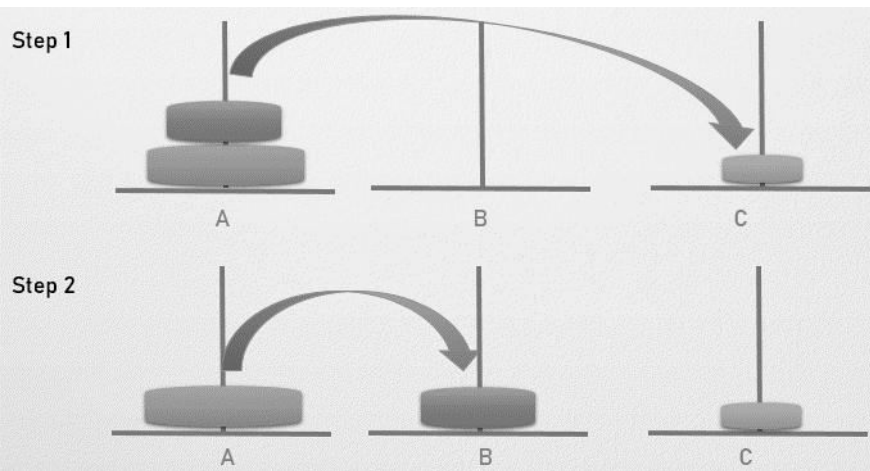
The Tower of Hanoi, is a mathematical problem which consists of three rods and multiple disks. Initially, all the disks are placed on one rod, one over the other in ascending order of size similar to a cone-shaped tower.

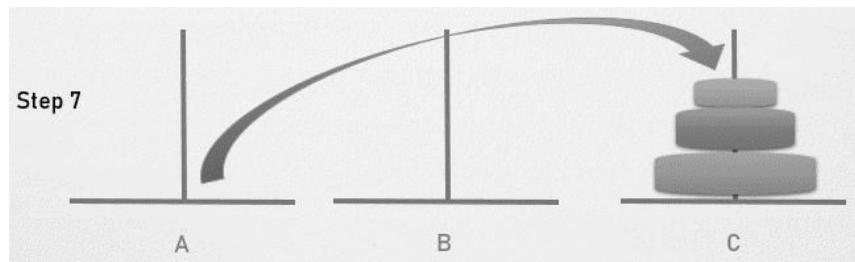


The objective of this problem is to move the stack of disks from the source to destination, following these rules:

1. Only one disk can be moved at a time.
2. Only the top disk can be removed.
3. No large disk can sit over a small disk.







Iterative Algorithm

1. At First Calculate the number of moves required i.e. " $2^n - 1$ " where " n " is number of discs.
2. If the number of discs i.e n is even then swap Destination Rod and Auxiliary Rod.
3. for $i = 1$ upto number of moves:
 - Check if " $i \bmod 3 == 1$ ":
 - Perform Movement of top disc between Source Rod and Destination Rod.
 - Check if " $i \bmod 3 == 2$ ":
 - Perform Movement of top disc between Source Rod and Auxiliary Rod.
 - Check if " $i \bmod 3 == 0$ ":
 - Perform Movement of top disc between Auxiliary Rod and Destination Rod.

Simulating Recursive Function using Stack

A recursive solution to a problem is often more expensive than a non-recursive solution, both in terms of time and space. Frequently, this expense is a small price to pay for the logical simplicity and self-documentation of the recursive solution. However, in a production program (such as a compiler, for example) that may be run thousands of times, the recurrent expense is a heavy burden on the system's limited resources.

Thus, a program may be designed to incorporate a recursive solution in order to reduce the expense of design and certification, and then carefully converted to a non-recursive version to be put into actual day-to-day use. As we shall see, in performing such a conversion it is often possible to identify parts of the implementation of recursion that are superfluous in a particular application and thereby significantly reduce the amount of work that the program must perform.

Suppose that we have the statement

route(x); where route is defined as a function by the header

route(a); x is referred to as an argument (of the calling function), and a is referred to as a parameter(of the called function).

What happens when a function is called? The action of calling a function may be divided into three parts:

1. Passing Arguments
2. Allocating and initializing local variables
3. Transferring control to the function.

1. **Passing arguments:** For a parameter in C, a copy of the argument is made locally within the function, and any changes to the parameter are made to that local copy. The effect to this scheme is that the original input argument cannot be altered. In this method, storage for the argument is allocated within the data area of the function.

2. **Allocating and initializing local variables:** After arguments have been passed, the local variables of the function are allocated. These local variables include all those declared directly in the function and any temporaries that must be created during the course of execution.

3. **Transferring control to the function:** At this point control may still not be passed to the function because provision has not yet been made for saving the return address. If a function is

given control, it must eventually restore control to the calling routine by means of a branch. However, it cannot execute that branch unless it knows the location to which it must return. Since this location is within the calling routine and not within the function, the only way that the function can know this address is to have it passed as an argument. This is exactly what happens. Aside from the explicit arguments specified by the programmer, there is also a set of implicit arguments that contain information necessary for the function to execute and return correctly. Chief among these implicit arguments is the return address. The function stores this address within its own data area. When it is ready to return control to the calling program, the function retrieves the return address and branches to that location. Once the arguments and the return address have been passed, control may be transferred to the function, since everything required has been done to ensure that the function can operate on the appropriate data and then return to the calling routine safely.

Summary

- A stack is a linear data structure in which allocation and deallocation are made in a last-in-first-out (LIFO) method.
- The basic operations of stack are inserting an element on the stack (push operation) and deleting an element from the stack (pop operation).
- Stacks are represented in main memory by using one-dimensional array or a singly linked list.
- To implement a stack structure, an array can be used as its storage structure. Each element of the stack occupies one array element. Static implementation of stack can be achieved using arrays.
- Stack is used to store return information in the case of function/procedure/subroutine calls. Hence, one would find a stack in architecture of any Central Processing Unit (CPU).
- In infix notation operators come in between the operands. An expression can be evaluated using stack data structure.

Keywords

LIFO: (Last In First Out) the property of a list such as stack in which the element which can be retrieved is the last element to enter it.

Pop: Stack operation retrieves a value from the stack.

Infix: Notation of an arithmetic expression in which operators come in between their operands.

Postfix: Notation of an arithmetic expression in which operators come after their operands.

Prefix: Notation of an arithmetic expression in which operators come before their operands.

Push: Stack operation which puts a value on the stack.

Stack: A linear data structure where insertion and deletion of elements can take place only at one end.

Self Assessment

1. Which method is followed by Stack?
 - A. FILO
 - B. LIFO
 - C. Both FILO and LIFO
 - D. None of above

2. Which is not stack operation?

- A. count()
- B. peek()
- C. getche()
- D. display()

3. Which is not part of stack?

- A. Overflow
- B. Enqueue
- C. Underflow
- D. None of above

4. To returns the element at the given position which function is used?

- A. isEmpty()
- B. isFull()
- C. peek()
- D. display()

5. Stack implementation is done using.....

- A. Array
- B. Linked list
- C. Both using array and linked list
- D. None of above

6. Parenthesis checker is used for

- A. Balanced Brackets
- B. Unbalanced Brackets
- C. Both balanced and unbalanced
- D. None of above

7. Which is incorrect statement?

- A. $(a+b*(c/d))$
- B. $[10+20*(6+7)]$
- C. $(x+y)/(c-d)$
- D. None of above

8. Which is not Sorting type

- A. Bubble
- B. Merge
- C. Insertion
- D. Linear

9. Tower of Hanoi is associated with....

- A. Queue
- B. Stack
- C. Array
- D. None of above

10. Arithmetic expressions can be represented as

- A. Infix notation
- B. Postfix notation
- C. Prefix notation
- D. All of above

11. Prefix notation can be represented as

- A. operator operand1 operand2
- B. operand1 operand2 operator
- C. operand1 operator operand1
- D. None of above

12. Postfix notation can be represented as

- A. operator operand1 operand2
- B. operand1 operator operand1
- C. operand1 operand2 operator
- D. None of above

13. Stack implementation is done using_____

- A. Statically
- B. Dynamically
- C. Both Statically and Dynamically
- D. None of above

14. Which is not stack operation?

- A. peek()
- B. pop()
- C. display()
- D. printf()

15. Underflow condition occur when stack is_____

- A. Full
- B. Empty
- C. Half
- D. None of above

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. C | 2. C | 3. B | 4. C | 5. C |
| 6. C | 7. B | 8. D | 9. B | 10. D |
| 11. A | 12. C | 13. C | 14. D | 15. B |

Review Questions

- 1 Explain how function calls may be implemented using stacks for return values.
- 2 What are the advantages of implementing a stack using dynamic memory allocation method?
- 3 Explain concept of tower of Hanoi.
- 4 What are the different methods for implementing stacks?
- 5 Give an example of push and pop operation using stack.
- 6 Write an algorithm to reverse an input string of characters using a stack.



Further Readings

Data Structures and Efficient Algorithms, Burkhard Monien, Thomas Ottmann, Springer.

Kruse Data Structure & Program Design, Prentice Hall of India, New Delhi

Mark Allen Weles: Data Structure & Algorithm Analysis in C Second Edition. Addison-Wesley publishing

RG Dromey, How to Solve it by Computer, Cambridge University Press.

Shi-kuo Chang, Data Structures and Algorithms, World Scientific

Sorenson and Tremblay: An Introduction to Data Structure with Algorithms.

Thomas H. Cormen, Charles E. Leiserson & Ronald L. Rivest: Introduction to Algorithms. Prentice-Hall of India Pvt. Limited, New Delhi

Timothy A. Budd, Classic Data Structures in C++, Addison Wesley.



Web Links

www.en.wikipedia.org

www.webopedia.com

<https://www.programiz.com/>

<https://www.javatpoint.com/data-structure-stack>

https://www.tutorialspoint.com/data_structures_algorithms/stack_algorithm.htm